Pointers and structs

Structs

What is a struct?

- A struct in C is a user-defined data type that groups related variables under one name
- It allows for better data organization and abstraction
- Similar to classes in object-oriented programming but without methods

Syntax:

```
#include <stdio.h>
typedef struct {
      int x;
      int y;
} Point;
```

```
int main(void)
{
      Point p1;

      p1.x = 3;
      p1.y = 2;

      return 0;
}
```

Pointers and structs

Pointers can be used to refer to structs dynamically

```
#include <stdio.h>
typedef struct {
      int x;
      int y;
} Point;
```

```
int main(void)
{
      Point p1;

      p1.x = 3;
      p1.y = 2;

      Point *p2;
      p2 = &p1;

      (*p2).x = 10;
      (*p2).y = 20;


      return 0;
}
```

Pointers and structs

Pointers can be used to refer to structs dynamically

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
      int x;
      int y;
} Point;
```

```
int main(void)
{
      Point p1;

      p1.x = 3;
      p1.y = 2;

      Point *p2;
      p2 = &p1;

      (*p2).x = 10;
      (*p2).y = 20;

      Point *p3;
      p3 = malloc(sizeof(Point));

      (*p3).x = 1;
      (*p3).y = 2;

      free(p3);

      return 0;
}
```

Pointers and the arrow operator

The arrow operator (->)

- Used for accessing struct members via pointers
- Equivalent to *(*ptr).member*

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
     int x;
     int y;
} Point;
```

```
int main(void)
{
     Point p1;

     p1.x = 3;
     p1.y = 2;

     Point *p2;
     p2 = &p1;

     p2->x = 10;
     p2->y = 20;

     Point *p3;
     p3 = malloc(sizeof(Point));

     p3->x = 1;
     p3->y = 2;

     free(p3);

     return 0;
}
```